

Université Hassan II de Casablanca
Faculté des Sciences Ben M'Sik
Département de Mathématiques et
Informatique



Plateforme E-Store Full-Stack

Réalisé par : EL MAZOUZY ISMAIL

Faculté des Sciences Ben M'Sik -

Département Informatique

Casablanca, Maroc

Année Universitaire : 2025 - 2026

REMERCIEMENTS

Je remercie Allah le Tout-Puissant de m'avoir donné la santé, la volonté et la patience nécessaires pour mener à bien ce modeste travail.

Au terme de ce projet, je tiens à exprimer ma profonde gratitude à toute l'équipe pédagogique de la **Faculté des Sciences Ben M'Sik**, et plus particulièrement aux enseignants du département de **Mathématiques et Informatique** pour la qualité de leur formation.

J'adresse mes sincères remerciements à mon encadrant, **M.Pr omar Zahour**, pour avoir suivi de près le déroulement de ce projet, pour ses conseils précieux et ses remarques constructives qui m'ont permis d'avancer. Ce fut un réel honneur de travailler sous sa direction.

Enfin, je tiens à remercier les membres du jury qui ont accepté d'évaluer ce travail et de m'honorer de leur présence. Mes remerciements s'étendent également à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce projet.

RÉSUMÉ

Ce projet présente la conception et la réalisation d'une plateforme de commerce électronique Full-Stack nommée "**E-Store**". L'application offre une solution complète permettant aux utilisateurs de naviguer à travers un catalogue de produits, de gérer un panier d'achat et d'effectuer des commandes de manière fluide. Elle intègre également un espace d'administration sécurisé permettant de gérer les stocks, les catégories et de suivre les ventes. Développée avec une architecture moderne, l'application utilise **Spring Boot** pour le backend et **React JS** pour le frontend, garantissant ainsi performance, sécurité et une expérience utilisateur optimisée.

ABSTRACT

In the current digital era, e-commerce has become an essential part of modern trade. This project describes the design and implementation of a Full-Stack e-commerce platform called "**E-Store**". The application provides a seamless shopping experience, allowing users to browse product catalogs, manage shopping carts, and place orders efficiently. Furthermore, it features a secure administrative dashboard for managing products, categories, and tracking customer orders. Built with a modern architecture, the system leverages **Spring Boot** for the backend and **React JS** for the frontend, ensuring high performance, security, and a responsive user interface.

TABLE DES MATIÈRES

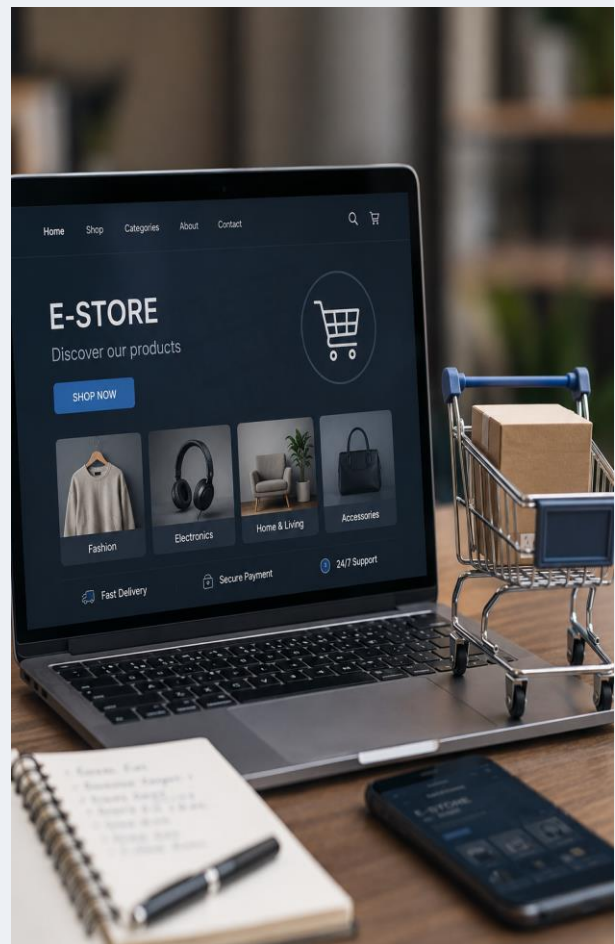
Dédicace & Remerciements	1
Résumé / Abstract	3
Liste des Figures & Tableaux	5
INTRODUCTION GÉNÉRALE	7
CHAPITRE 1 : PRÉSENTATION DU PROJET	9
1. Contexte et Problématique	10
2. Objectifs de l'application E-Store	12
3. Méthodologie de travail	14
CHAPITRE 2 : ANALYSE DES BESOINS	16
1. Besoins fonctionnels (Client & Admin)	17
2. Besoins non-fonctionnels	22
3. Identification des acteurs	25
CHAPITRE 3 : CONCEPTION ET MODÉLISATION	28
1. Architecture globale du système	29
2. Modélisation UML (Cas d'utilisation, Séquence, Classe)	31
3. Modélisation de la base de données (MCD)	40
CHAPITRE 4 : ENVIRONNEMENT TECHNIQUE	45
1. Technologies Backend (Spring Boot)	46
2. Technologies Frontend (React JS)	50
3. Outils de développement et Tests	54
CHAPITRE 5 : RÉALISATION ET RÉSULTATS	58
1. Structure technique du projet	59
2. Présentation des interfaces (Captures d'écran)	62
3. Sécurité et gestion des accès	68
CONCLUSION GÉNÉRALE	72
BIBLIOGRAPHIE	74
ANNEXES	75

INTRODUCTION GÉNÉRALE

L'évolution rapide des technologies de l'information a radicalement transformé le secteur du commerce. Aujourd'hui, la vente en ligne est devenue un pilier incontournable de l'économie mondiale. Dans ce contexte, le développement d'une plateforme e-commerce ne se limite plus à la simple présentation de produits, mais englobe la gestion complexe des données, la sécurité des transactions et l'optimisation de l'expérience utilisateur.

L'objectif de ce projet, intitulé "**E-STORE**", est de concevoir et réaliser une application web full-stack performante. Ce travail nous permet de mettre en pratique nos connaissances académiques en utilisant des technologies modernes telles que **Spring Boot** pour le backend et **React JS** pour le frontend.

Le présent rapport est structuré en plusieurs chapitres : le premier présente le contexte du projet, le second traite l'analyse des besoins, le troisième est dédié à la conception, le quatrième détaille les outils utilisés, et le dernier expose la réalisation finale.



En somme, ce travail ne se limite pas à une simple réalisation technique, mais constitue une véritable immersion dans le cycle de vie d'un projet informatique complexe. Il met en lumière l'importance d'une planification rigoureuse et d'une conception modulaire pour assurer la pérennité d'une application de commerce électronique.

À travers les pages qui suivent, nous détaillerons chaque étape de cette aventure technologique. Nous commencerons par définir le cadre dans lequel s'inscrit ce projet, en analysant les motivations qui nous ont poussés à choisir le domaine de l'e-commerce, ainsi que les défis spécifiques auxquels nous avons dû faire face pour garantir une expérience utilisateur de haut niveau. Cette introduction sert ainsi de pont entre nos acquis théoriques au sein de la **Faculté des Sciences Ben M'Sik** et les exigences concrètes du monde professionnel du développement web.

CHAPITRE 1 : PRÉSENTATION DU PROJET

1. Contexte et Problématique

1.1. Contexte Général

Dans un monde où la transformation numérique redéfinit les habitudes de consommation, le commerce électronique s'est imposé comme un secteur stratégique à l'échelle mondiale. Selon les estimations, le chiffre d'affaires mondial de l'e-commerce a dépassé les **5 000 milliards de dollars en 2024**, avec une croissance annuelle qui ne montre aucun signe de ralentissement. Cette évolution profonde est portée par la démocratisation de l'accès à Internet, la généralisation des smartphones et la confiance croissante des consommateurs envers les achats en ligne.

Dans ce contexte, les entreprises — qu'elles soient de grande envergure ou de petite et moyenne taille — ressentent un besoin pressant de disposer de plateformes e-commerce **robustes, modulaires, évolutives et sécurisées**. Cependant, les solutions existantes sur le marché présentent souvent des limites importantes : elles sont soit trop rigides et non personnalisables (comme certains CMS), soit trop complexes à maintenir pour des équipes de taille réduite. La grande majorité de ces plateformes fonctionnent comme des **boîtes noires**, rendant toute adaptation technique difficile, voire impossible.

Par ailleurs, du point de vue académique, le développement d'une plateforme e-commerce from scratch représente un défi technologique complet, mobilisant des compétences transversales : conception de bases de données relationnelles et non-relationnelles, développement d'APIs RESTful, gestion de l'état applicatif côté frontend, sécurisation des accès et optimisation des performances. C'est précisément dans ce cadre que s'inscrit le projet **E-Store**.

1.2. Problématique

Face aux limites des solutions existantes et à la complexité inhérente au développement d'une application e-commerce complète, la problématique centrale de notre projet se formule ainsi :

« Comment concevoir et développer une plateforme e-commerce Full Stack complète, maintenable et sécurisée, en appliquant les meilleures pratiques du génie logiciel moderne, notamment l'architecture découplée, le pattern MVC et la persistance hybride SQL/NoSQL ? »

Cette problématique soulève plusieurs questions techniques :

- Comment structurer le code pour garantir la **séparation des responsabilités** entre Frontend, Backend et Base de données ?
- Comment assurer la **cohérence des données** lors d'opérations critiques comme la validation d'une commande et la mise à jour du stock ?
- Comment offrir une **expérience utilisateur fluide et réactive** tout en maintenant une architecture backend solide ?
- Comment gérer efficacement des données de **nature différente** : relationnelles pour les transactions et non-relationnelles pour les avis clients ?

2. Objectifs de l'Application E-Store

2.1. Objectif Global

L'objectif principal du projet E-Store est de concevoir et réaliser une **plateforme de commerce électronique Full Stack complète**, allant de l'inscription de l'utilisateur jusqu'à la confirmation d'une commande, en passant par la navigation dans le catalogue, la gestion du panier et l'administration des produits.

2.2. Objectifs Fonctionnels

Domaine	Objectif
Authentification	Permettre l'inscription, la connexion et la gestion du profil utilisateur
Catalogue	Afficher les produits par catégorie, avec recherche dynamique
Panier	Gérer l'ajout, la modification et la suppression d'articles
Commandes	Créer des commandes avec décrémentation automatique du stock
Avis clients	Permettre la notation et les commentaires sur les produits
Administration	Fournir un panneau de gestion CRUD complet pour les produits

2.3. Objectifs Techniques

Sur le plan technique, le projet vise à atteindre les standards suivants :

- **Architecture découplée** : séparer strictement le Frontend (React.js) du Backend (Spring Boot), communiquant uniquement via des APIs REST.
- **Architecture en couches** : respecter le pattern MVC avec les couches Controller, Service, Repository et DTO.
- **Persistance hybride** : combiner MySQL pour les données transactionnelles et MongoDB pour les avis clients (approche Polyglot Persistence).
- **Qualité du code** : appliquer les principes SOLID, utiliser le pattern DTO pour découpler les entités JPA des réponses API, et gérer les exceptions de manière centralisée.
- **Testabilité** : valider l'ensemble des APIs REST avec Postman avant toute intégration Frontend.
- **Expérience utilisateur** : offrir une interface responsive, réactive et intuitive avec des retours visuels immédiats.

3. Méthodologie de Travail

3.1. Approche Adoptée

Le projet E-Store a été conduit selon une **méthodologie itérative et incrémentale**, inspirée des principes Agile. Cette approche nous a permis de livrer progressivement des modules fonctionnels, de tester chaque composant indépendamment et d'adapter notre développement en fonction des difficultés rencontrées.

Le développement a été organisé en **cinq phases successives** :

Phase 1 — Analyse des besoins (*Semaine 1*)

Identification des besoins fonctionnels et non-fonctionnels, définition des acteurs du système (utilisateur, administrateur), et rédaction des spécifications.

Phase 2 — Conception et Modélisation (*Semaine 2*)

Modélisation de l'architecture générale, conception du schéma de la base de données, réalisation des diagrammes UML (cas d'utilisation, séquence, classes) et définition des contrats d'API.

Phase 3 — Développement Backend (*Semaines 3 et 4*)

Mise en place du projet Spring Boot, création des entités JPA, implémentation des repositories, services et controllers, configuration de MySQL et MongoDB, et validation de chaque endpoint via Postman.

Phase 4 — Développement Frontend (Semaines 5 et 6)

Création du projet React.js, développement des composants et pages, intégration avec le Backend via Axios, gestion du routing avec React Router, et implémentation de l'interface d'administration.

Phase 5 — Tests et Validation (Semaine 7)

Tests fonctionnels complets, correction des bugs identifiés, optimisation des performances et rédaction du rapport.

3.2. Tableau Récapitulatif de la Méthodologie

Phase	Activités principales	Outils utilisés	Durée
Analyse	Spécification des besoins, identification des acteurs	—	1 semaine
Conception	UML, MCD, architecture, contrats API	Draw.io	1 semaine
Backend	Spring Boot, JPA, REST APIs, MongoDB	IntelliJ IDEA, Postman	2 semaines
Frontend	React.js, Axios, CSS, Routing	VS Code, npm	2 semaines
Tests	Validation fonctionnelle, corrections	Postman, navigateur	1 semaine

SCRUM PROCESS

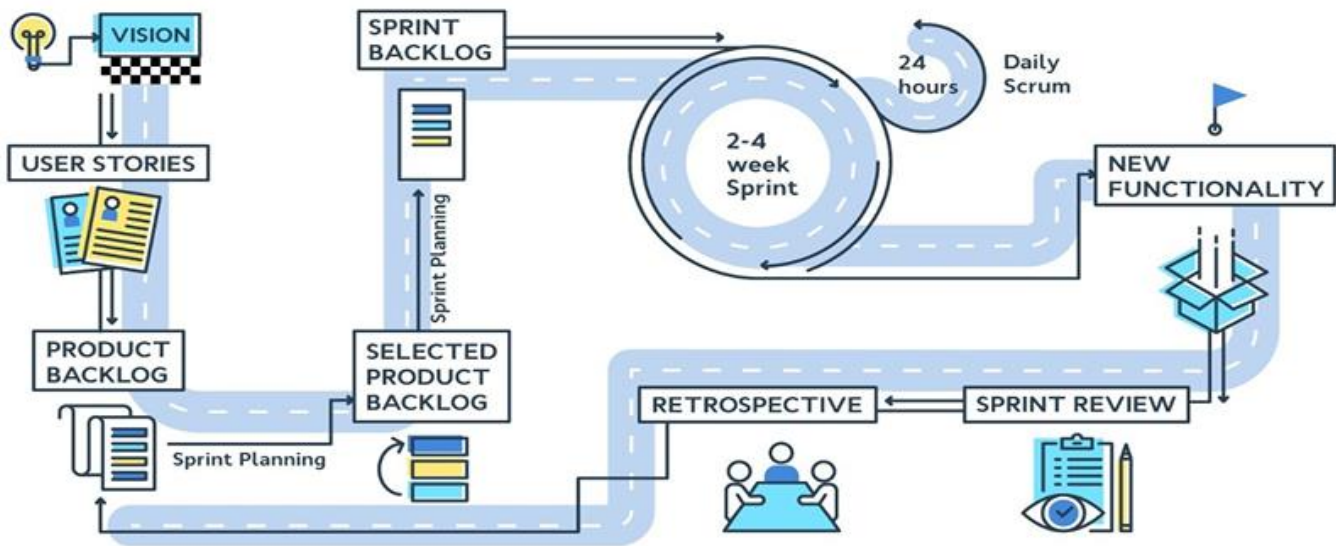


Figure 1 : Scrum Process - Méthodologie itérative du projet

3.3. Organisation du Travail

Afin de garantir un développement structuré et traçable, le projet a été versionné avec **Git**, permettant de maintenir un historique complet de toutes les modifications apportées au code. Chaque domaine métier (Customer, Catalog, Inventory, Shopping, Billing, Review) a été développé indépendamment, testés séparément, puis intégré progressivement dans l'application complète.

Cette organisation modulaire a constitué un avantage majeur lors du débogage : la localisation d'une erreur dans un domaine n'impactait pas les autres modules déjà validés.

4. Présentation de l'Application E-Store

4.1. Description Générale

E-Store est une plateforme de commerce électronique web complète, développée with une architecture trois-tiers strictement découplée. Elle offre aux utilisateurs un cycle d'achat complet et intuitif, depuis la consultation du catalogue jusqu'à la confirmation de commande, en passant par la gestion du panier.

La plateforme se distingue par les caractéristiques suivantes :

- **Interface moderne** : design épuré, responsive et réactif développé avec React.js.
- **Backend robuste** : API REST développée avec Spring Boot 4 et Java 17.

- **Base de données hybride** : MySQL pour les données transactionnelles, MongoDB pour les avis.
- **Panneau d'administration** : gestion complète des produits, catégories et stocks directement depuis l'interface web.

4.2. Acteurs du Système

L'application E-Store distingue deux types d'acteurs principaux :

Acteur	Description	Droits
Utilisateur non authentifié	Visiteur du site	Consulter le catalogue, s'inscrire, se connecter
Utilisateur authentifié	Client enregistré	Panier, commandes, avis, profil
Administrateur	Gestionnaire de la plateforme	CRUD produits, catégories, stocks

4.3. Périmètre Fonctionnel

Les tableaux ci-dessous résume les grandes fonctionnalités couvertes par l'application E-Store :

Architecture Fonctionnelle – Vue d'Ensemble

Module Authentification

Action	Méthode	Endpoint
Inscription	POST	/api/auth/register
Connexion	POST	/api/auth/login

Module Catalogue

Action	Méthode	Endpoint
--------	---------	----------

Liste des produits	GET	/api/products
Recherche par mot-clé	GET	/api/products?search=
Liste des catégories	GET	/api/categories
Créer un produit	POST	/api/products
Modifier un produit	PUT	/api/products/{id}
Supprimer un produit	DELETE	/api/products/{id}

Module Panier

Action	Méthode	Endpoint
Consulter le panier	GET	/api/cart/{userId}
Ajouter un produit	POST	/api/cart/add
Modifier la quantité	PUT	/api/cart/update
Supprimer un article	DELETE	/api/cart/remove/{id}

Module Commandes

Action	Méthode	Endpoint
Créer une commande	POST	/api/orders
Historique des commandes	GET	/api/orders/user/{id}

Module Inventaire

Action	Méthode	Endpoint
--------	---------	----------

Consulter le stock	GET	/api/inventory/{id}
Mettre à jour le stock	PUT	/api/inventory/{id}

Module Avis Clients (*MongoDB*)

Action	Méthode	Endpoint
Soumettre un avis	POST	/api/reviews
Consulter les avis	GET	/api/reviews/product/{id}

CHAPITRE 2 : ANALYSE DES BESOINS ET SPÉCIFICATIONS

1. Introduction au Chapitre

L'analyse des besoins constitue une phase critique du cycle de développement. Elle vise à transformer les objectifs généraux du projet en spécifications détaillées et mesurables, accessibles tant aux parties techniques qu'aux stakeholders métier. Cette phase jette les bases sur lesquelles reposera toute la conception et la réalisation ultérieure.

Pour le projet E-Store, cette analyse a impliqué une décomposition systématique des exigences en besoins fonctionnels précis et en contraintes non-fonctionnelles explicites.

2. Besoins Fonctionnels Détaillés

2.1. Module d'Authentification et de Gestion Utilisateur

Besoin BF-AUTH-001 : Inscription Utilisateur

Un visiteur non authentifié doit pouvoir créer un compte utilisateur en fournissant : nom complet, adresse électronique (unique), mot de passe (minimum 8 caractères), adresse de livraison, et téléphone. Le système doit vérifier l'unicité de l'email, hasher le mot de passe avant stockage, envoyer un email de confirmation, et rediriger vers la connexion.

Besoin BF-AUTH-002 : Connexion Utilisateur

Un visiteur doit pouvoir se connecter via email et mot de passe. Le système doit vérifier les identifiants et générer un token JWT valide pour 24 heures.

Besoin BF-AUTH-003 : Gestion du Profil

Un utilisateur authentifié doit pouvoir consulter son profil, modifier ses informations, changer son mot de passe (avec vérification de l'ancien), et mettre à jour son adresse de livraison.

Besoin BF-AUTH-004 : Déconnexion

Un utilisateur authentifié doit pouvoir se déconnecter (suppression du token local), recevoir un message de confirmation, et être redirigé vers la page d'accueil.

2.2. Module de Catalogue et Gestion des Produits

Besoin BF-CAT-001 : Affichage du Catalogue

Le système doit afficher tous les produits disponibles avec miniature, nom, catégorie, prix, note moyenne, stock, et bouton d'ajout au panier. Les produits doivent être

affichés avec pagination (12 produits par page) et triés par pertinence, prix, note, ou date de création.

Besoin BF-CAT-002 : Recherche Dynamique

Les utilisateurs doivent pouvoir rechercher par mot-clé (nom et description), catégorie, gamme de prix, et notation minimale. La recherche doit être en temps réel avec debounce de 300ms.

Besoin BF-CAT-003 : Détail Produit

Chaque produit doit afficher images haute résolution, description détaillée, caractéristiques, stock, avis clients, et produits similaires (recommandations).

Besoin BF-CAT-004 : Gestion Administrateur des Produits

Les administrateurs doivent pouvoir créer, modifier, supprimer les produits, importer des produits en masse (CSV), télécharger les images, et activer/désactiver les produits.

2.3. Module Panier d'Achat

Besoin BF-CART-001 : Gestion du Panier Persistant

Le système doit maintenir un panier pour chaque utilisateur avec stockage en base de données (pas uniquement localStorage). Le panier doit synchroniser en temps réel entre sessions, calculer automatiquement le total, gérer les quantités, et permettre la suppression d'articles.

Besoin BF-CART-002 : Ajout au Panier

Lors de l'ajout d'un produit, le système doit vérifier la disponibilité du stock, augmenter la quantité si le produit existe déjà, afficher une notification de confirmation, mettre à jour le compteur du panier, et afficher le prix actualisé.

Besoin BF-CART-003 : Modification du Panier

L'utilisateur doit pouvoir modifier la quantité (avec validation stock), appliquer des codes promotionnels, consulter le détail (prix unitaire, sous-total, taxes), et voir les frais de livraison calculés.

Besoin BF-CART-004 : Transfert d'Images Produit

Le système doit transférer les images des produits du panier, afficher les miniatures sans erreur, supporter les images manquantes (image par défaut), et optimiser la taille des images pour le web.

2.4. Module Commandes

Besoin BF-ORDER-001 : Création de Commande

À partir du panier, l'utilisateur doit valider le panier (vérifier quantités et stock), sélectionner l'adresse de livraison, choisir le mode de livraison (standard, express), voir un résumé avant confirmation. Le système doit créer l'enregistrement de commande,

créer les lignes de commande, décrémenter automatiquement le stock, générer un numéro de confirmation, vider le panier, envoyer une confirmation par email, et afficher le numéro de suivi.

Besoin BF-ORDER-002 : Historique des Commandes

Chaque utilisateur doit pouvoir consulter la liste de ses commandes avec dates, l'état de chaque commande, le détail complet (articles, montants, adresse), et la date estimée de livraison.

Besoin BF-ORDER-003 : Gestion Administrateur des Commandes

Les administrateurs doivent pouvoir voir toutes les commandes, filtrer par utilisateur/date/état, modifier l'état d'une commande, générer des rapports de ventes, et analyser les tendances d'achat.

3. Besoins Non-Fonctionnels

3.1. Exigences de Performance

Ré-NFR-PERF-001 : Temps de Réponse

Les requêtes de consultation doivent répondre en < 500ms. Les pages doivent charger en < 3 secondes (Time to Interactive). L'ajout au panier doit être instantané (< 100ms).

Ré-NFR-PERF-002 : Scalabilité

L'application doit supporter 1000 utilisateurs simultanés. L'architecture doit permettre une augmentation de capacité. Les requêtes doivent être optimisées (utiliser les indexes).

Ré-NFR-PERF-003 : Caching

Implémenter un cache pour le catalogue (30 minutes TTL). Utiliser le localStorage pour le panier côté frontend. Implémenter la pagination des résultats.

3.2. Exigences de Sécurité

Ré-NFR-SEC-001 : Authentification et Autorisation

Implémenter JWT pour la gestion des sessions. Protéger les endpoints administrateurs. Valider les tokens à chaque requête sensible. Gérer l'expiration des tokens.

Ré-NFR-SEC-002 : Protection des Données

Hasher les mots de passe avec bcrypt ou Argon2. Valider et nettoyer toutes les entrées utilisateur. Implémenter la protection CSRF. Utiliser HTTPS en production.

Ré-NFR-SEC-003 : Conformité

Respecter les lois de protection des données (RGPD). Implémenter le droit à l'oubli. Fournir un export de données utilisateur.

3.3. Exigences de Disponibilité et Maintenabilité

Ré-NFR-MAINT-001 : Qualité du Code

Suivre les conventions de codage Java et JavaScript. Maintenir une cohésion élevée et un couplage bas. Utiliser la documentation dans le code. Créer des tests unitaires (couverture > 70%).

Ré-NFR-MAINT-002 : Logs et Monitoring

Logger tous les événements importants. Tracer les erreurs avec stack traces. Implémenter les logs structurés (JSON). Monitorer les performances en production.

Ré-NFR-MAINT-003 : Déploiement

Utiliser Docker pour la conteneurisation. Implémenter l'intégration continue (CI/CD). Automatiser les tests avant déploiement. Supporter le déploiement zero-downtime.

3.4. Exigences d'Accessibilité et UX

Ré-NFR-UX-001 : Responsive Design

Supporter les écrans de 320px (mobile) à 4K. Afficher 1 colonne sur mobile, 2 sur tablette, 4 sur desktop. Adapter les interfaces tactiles et souris. Tester sur iOS et Android.

Ré-NFR-UX-002 : Accessibilité

Respecter les normes WCAG 2.1 niveau AA. Fournir les attributs alt pour les images. Assurer une navigation au clavier. Utiliser des contrastes adéquats.

Ré-NFR-UX-003 : Internationalisation

Supporter plusieurs langues (français, anglais, arabe). Gérer les formats de monnaie et date. Traduire tous les textes.

4. Identification des Acteurs et Cas d'Utilisation

4.1. Acteurs du Système

Acteur 1 : Visiteur (Utilisateur non authentifié)

Peut consulter le catalogue, effectuer une recherche, créer un compte (s'inscrire), et se connecter.

Acteur 2 : Client (Utilisateur authentifié)

Possède tous les droits du visiteur. Peut ajouter des articles au panier, valider un panier et créer une commande, consulter l'historique de ses commandes, laisser des avis sur les produits, et gérer son profil.

Acteur 3 : Administrateur

Possède tous les droits du client. Peut créer, modifier, supprimer des produits, gérer les catégories, consulter et modifier les stocks, voir toutes les commandes, et modérer les avis.

4.2. Diagramme de Cas d'Utilisation

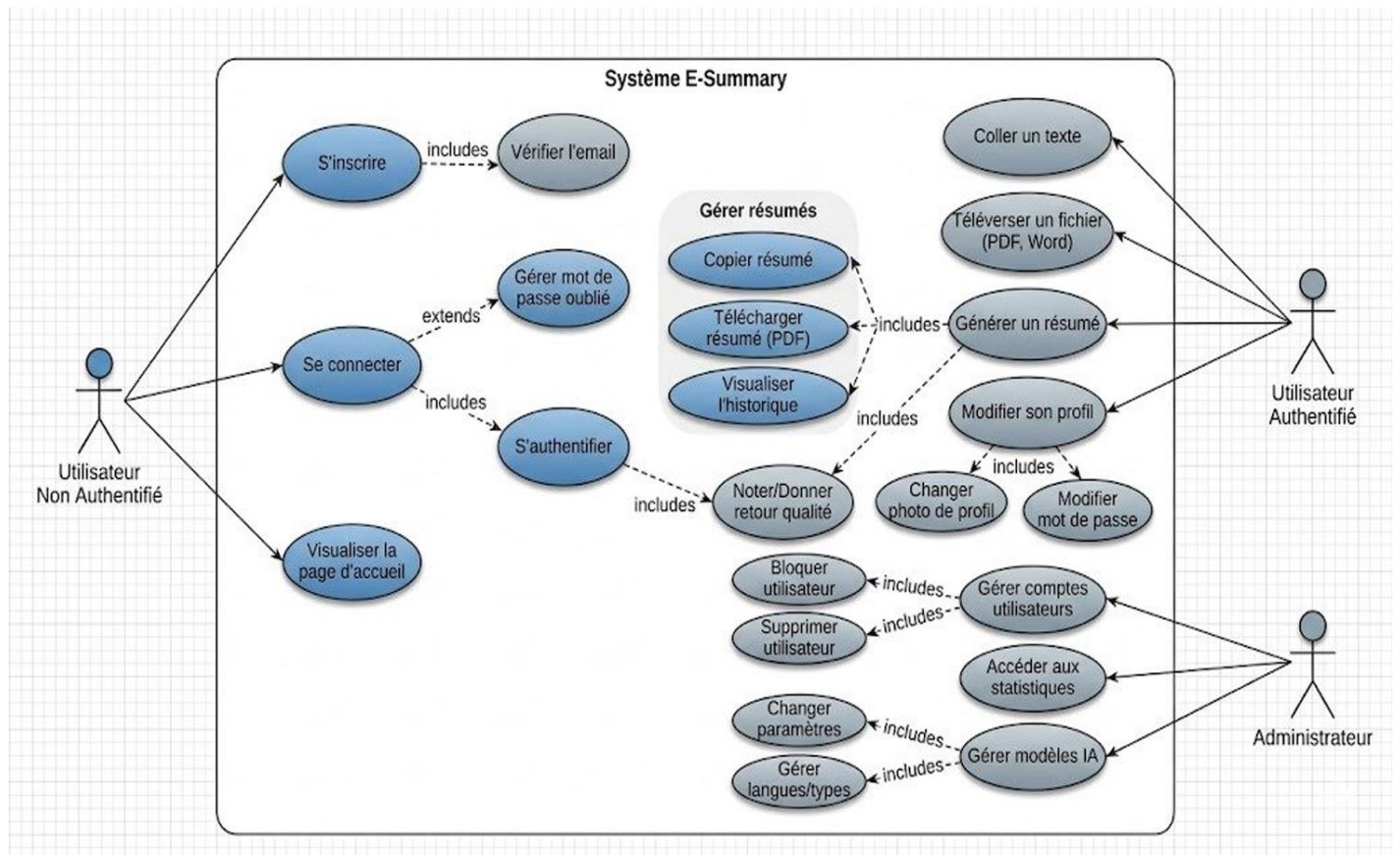


Figure 2 : Diagramme UML des Cas d'Utilisation - Vue d'ensemble du système

Cas d'Utilisation Principal : Effectuer un Achat

Acteur Primaire : Client

Préconditions : L'utilisateur est authentifié

Flux Principal :

1. L'utilisateur navigué dans le catalogue
2. L'utilisateur recherche un produit
3. L'utilisateur consulte les détails du produit
4. L'utilisateur ajoute le produit au panier
5. L'utilisateur accède à la page du panier
6. L'utilisateur valide le panier
7. L'utilisateur sélectionne l'adresse de livraison

8. L'utilisateur choisit le mode de livraison
9. L'utilisateur confirme la commande
10. Le système crée la commande
11. Le système décrémente le stock automatiquement
12. Le système vide le panier
13. Le système envoie une confirmation par email
14. Le système affiche le numéro de suivi

Postconditions : La commande est enregistrée, le stock est mis à jour

CHAPITRE 3 : CONCEPTION ET MODÉLISATION

1. Introduction à la Conception Architecturale

La phase de conception consiste à transformer les spécifications élaborées au chapitre précédent en modèles architecturaux et techniques détaillés. Cette phase est cruciale pour garantir que l'implémentation qui suivra sera robuste, scalable et maintenable.

Pour le projet E-Store, la conception s'est déroulée selon une approche top-down : nous avons d'abord défini l'architecture globale du système, puis décomposé celle-ci en couches fonctionnelles, avant de concevoir les schémas de données et les interfaces.

2. Architecture Globale du Système

2.1. Architecture Trois-Tiers

E-Store s'appuie sur une architecture trois-tiers classique, où chaque tiers est strictement séparé et communique via des interfaces bien définies. Cette architecture offre plusieurs avantages majeurs :

- **Séparation** des préoccupations : chaque couche a une responsabilité unique
- **Scalabilité** : le backend peut être déployé sur plusieurs serveurs
- **Testabilité** : chaque couche peut être testée indépendamment
- **Maintenabilité** : les modifications dans une couche n'impactent pas les autres
- **Flexibilité** : le frontend et le backend peuvent évoluer indépendamment

Couche Présentation (Frontend) React.js / JavaScript / CSS

- Composants réutilisables
- Gestion d'état (useState, useContext)
- Routing avec React Router
- Appels HTTP avec Axios

Couche Métier (Backend) Spring Boot / Java 17 / Maven

- Contrôleurs (gestion des requêtes HTTP)
- Services (logique métier)
- Repositories (accès aux données)
- DTOs (transfert de données)
- Entités (modélisation des données)

Couche Données (Persistance) MySQL + MongoDB

- Données transactionnelles (MySQL)
- Données non-relationnelles (MongoDB)

2.2. Architecture en Couches du Backend

Le backend Spring Boot suit le pattern MVC avec des couches bien définies. Chaque couche a une responsabilité spécifique et ne doit pas être couplée directement aux autres :

Couche Controller

Les contrôleurs gèrent les requêtes HTTP entrantes et les réponses sortantes. Ils reçoivent les requêtes, en extraient les paramètres, délèguent à la couche service, convertissent les objets métier en DTOs, et retournent les réponses HTTP avec les codes appropriés.

Couche Service

La couche service contient toute la logique métier. Elle implémente les règles de gestion métier, coordonne les opérations complexes, gère les transactions, et valide les données métier.

Couche Repository (Data Access)

Les repositories gèrent l'accès aux données. Avec Spring Data JPA, nous utilisons des interfaces qui fournissent automatiquement l'implémentation pour les opérations CRUD et les requêtes personnalisées.

Couche Entity

Les entités représentent les objets du domaine persistants. Elles sont annotées avec JPA pour automatiser le mappage entre les objets Java et les tables de la base de données.

Data Transfer Objects (DTOs)

Les DTOs découplent les entités JPA des réponses API. Cela permet de modifier la structure interne de la base de données sans impacter les clients.

Avantages de cette architecture en couches :

- **Responsabilité unique** : chaque classe a une seule raison de changer
- **Testabilité** : services testables sans base de données via des mocks
- **Réutilisabilité** : les services peuvent être utilisés par plusieurs contrôleurs
- **Maintenabilité** : changement de persistance (SQL → NoSQL) ne touche que les repositories

2.3. Polyglot Persistence : SQL + NoSQL

E-Store combine deux technologies de persistance pour bénéficier des avantages de chacune :

MySQL pour les Données Transactionnelles

MySQL est utilisé pour les données qui nécessitent l'ACID compliance :

- Utilisateurs et authentification

- Produits et catégories
- Commandes et historique
- Inventaire et mouvements de stock

Ces données bénéficient des transactions ACID, des contraintes d'intégrité et de la normalisation pour éviter les anomalies.

MongoDB pour les Avis

MongoDB stocke les avis clients avec un schéma flexible. Cela permet d'ajouter de nouveaux champs (images, tags, etc.) sans migrer le schéma.

Justification du Polyglot Persistence :

- **Flexibilité du schéma** : les avis peuvent avoir des attributs variables
- **Performance** : MongoDB excelle pour les lectures denormalisées
- **Scalabilité** : MongoDB supporte bien les opérations d'agrégation
- **Séparation** : les avis ne sont pas transactionnels (non-critiques)

3. Modélisation UML Détaillée

3.1. Diagramme de Cas d'Utilisation Figure

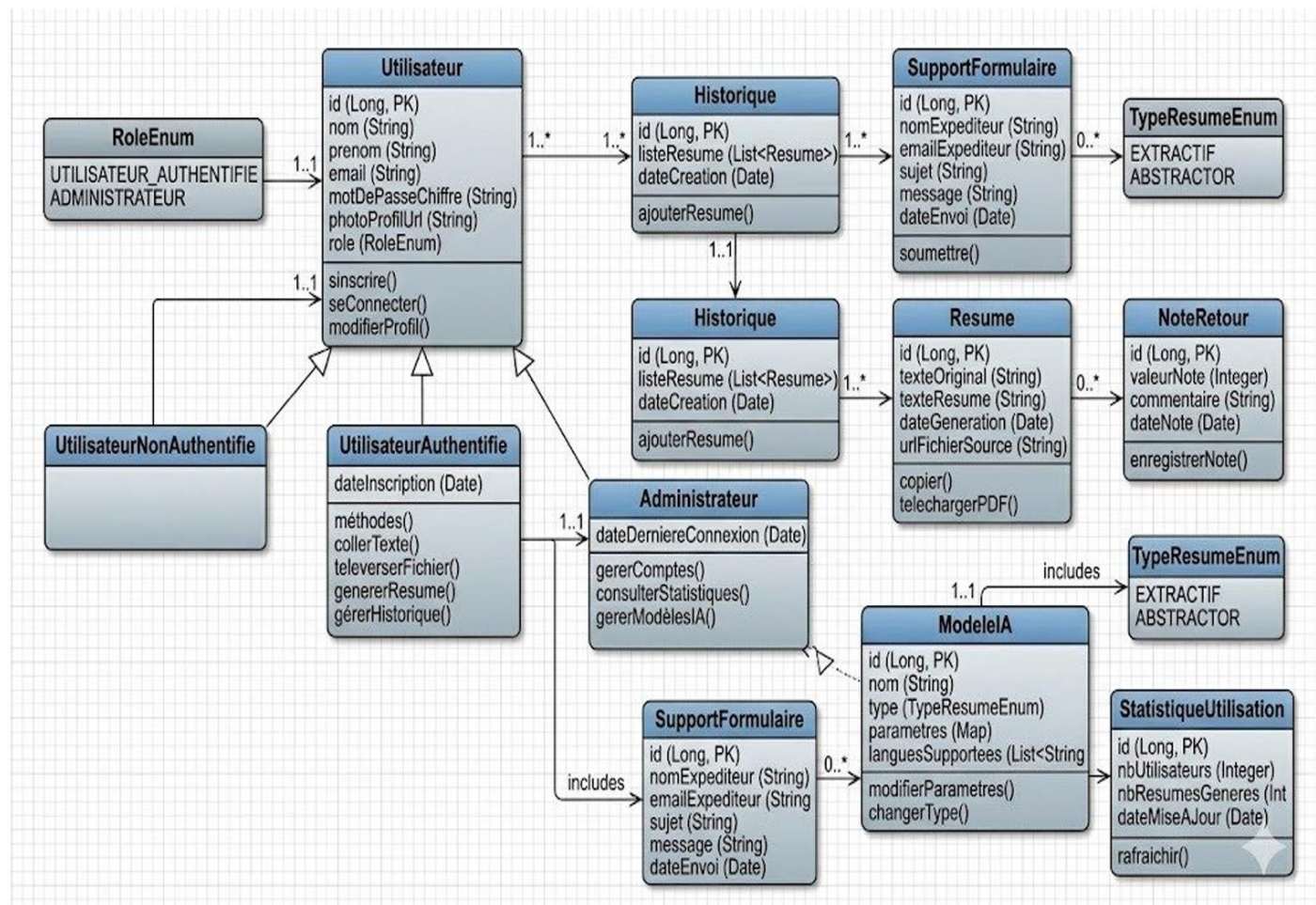


Figure 3 : Diagramme de Classes - Entités principales du système

3.2. Principaux Agregats Métier

Les principaux agregats du domaine sont les suivants :

Agrégat Utilisateur

Entités : User, Address, Role

Responsabilité : Gérer l'authentification, le profil, et les adresses de livraison.

Maintenir les rôles et droits d'accès.

Agrégat Catalogue

Entités : Category, Product, Inventory

Responsabilité : Gérer le catalogue des produits, les catégories, et les stocks. Supporter la recherche et le filtrage.

Agrégat Commande

Entités : Order, OrderItem

Responsabilité : Gérer le cycle de vie des commandes, de leur création à leur livraison.

Maintenir l'historique.

Agrégat Panier

Entités : Cart, CartItem

Responsabilité : Gérer le panier d'achat de l'utilisateur. Supporter l'ajout, la modification, et la suppression d'articles.

Agrégat Avis (MongoDB)

Collections : Review

Responsabilité : Stocker et récupérer les avis clients. Calculer les statistiques d'avis (moyenne, distribution).

3.3. Diagrammes de Séquence

3.3.1. Authentification / Connexion

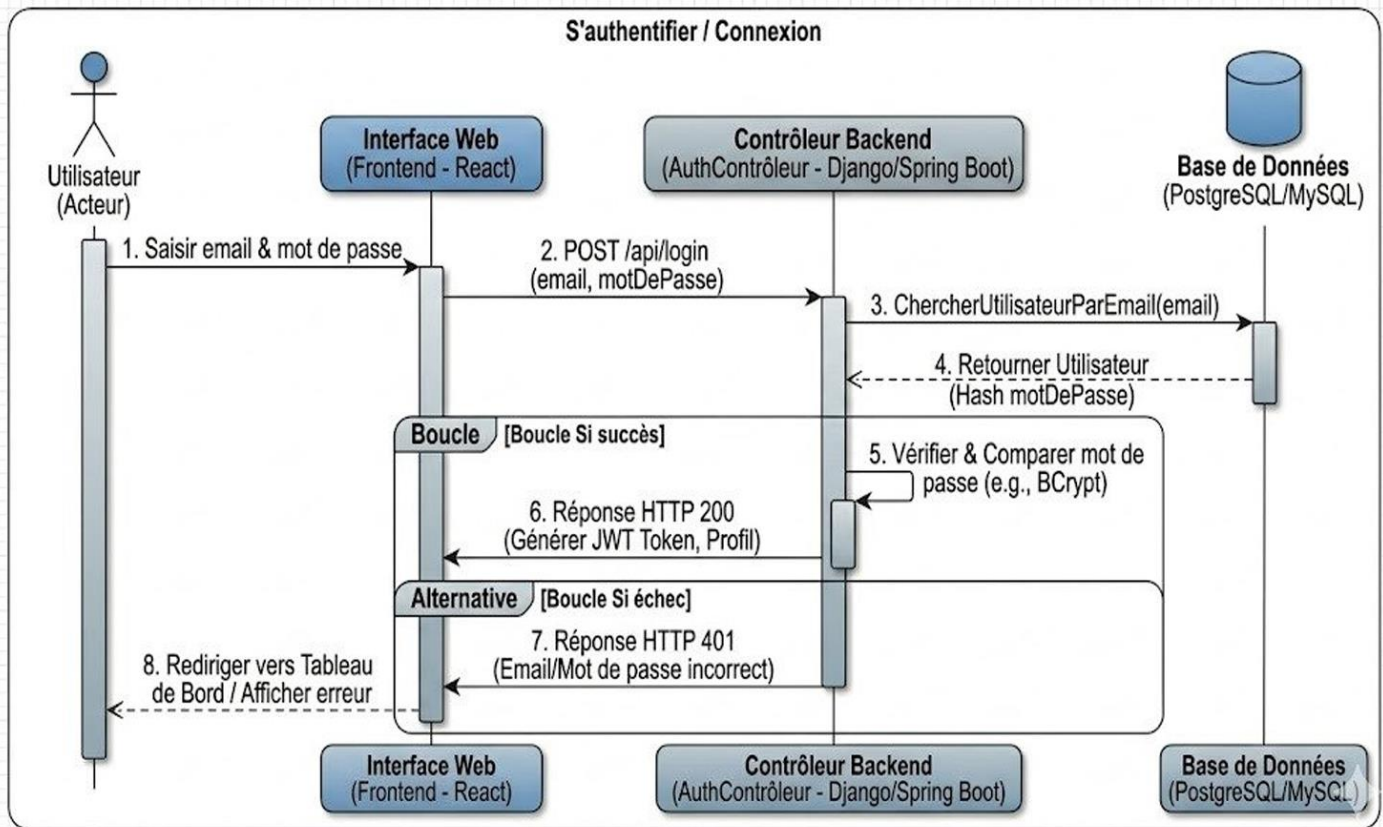


Figure 4 : Diagramme de Séquence - Processus d'Authentification

3.3.2. Gestion de l'Histoire

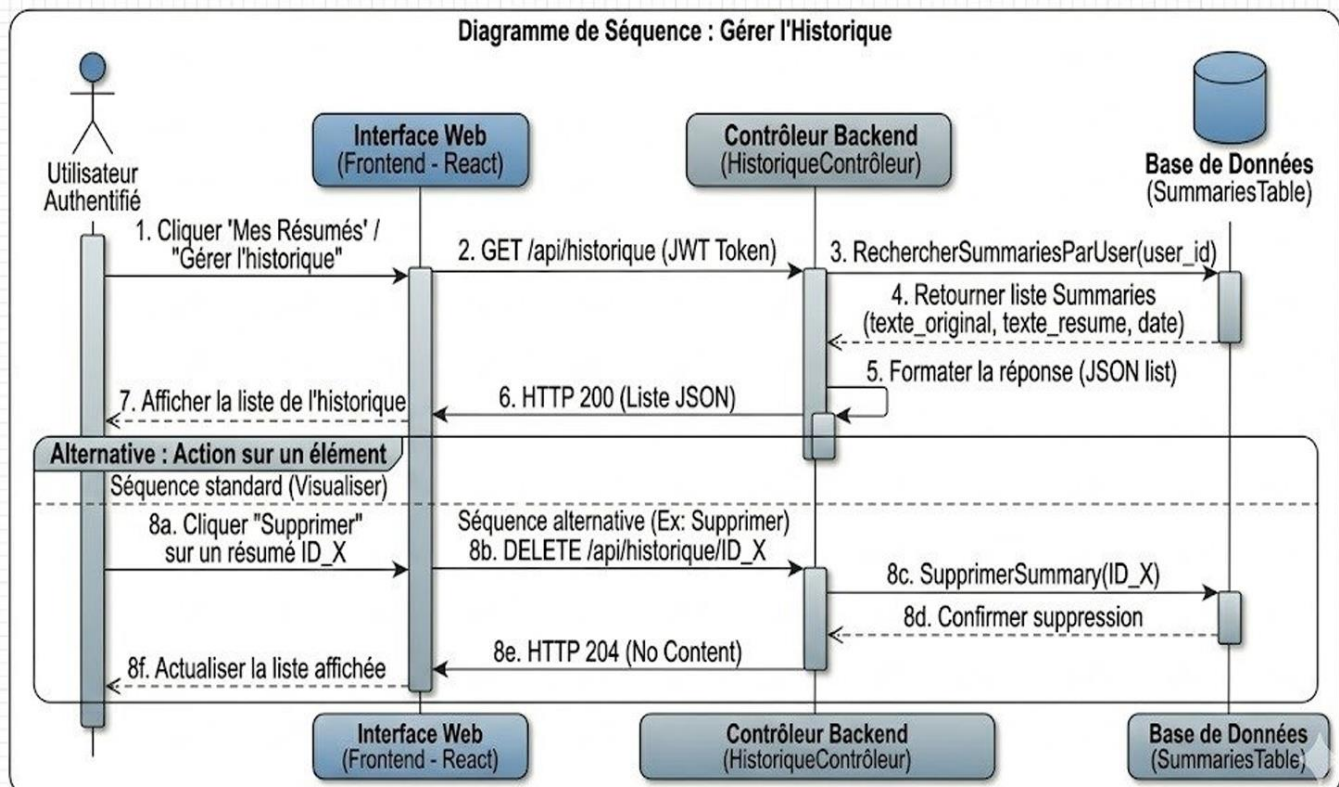


Figure 5 : Diagramme de Séquence - Consultation de l'Histoire des Commandes

3.3.3. Récupération de Mot de Passe Oublié

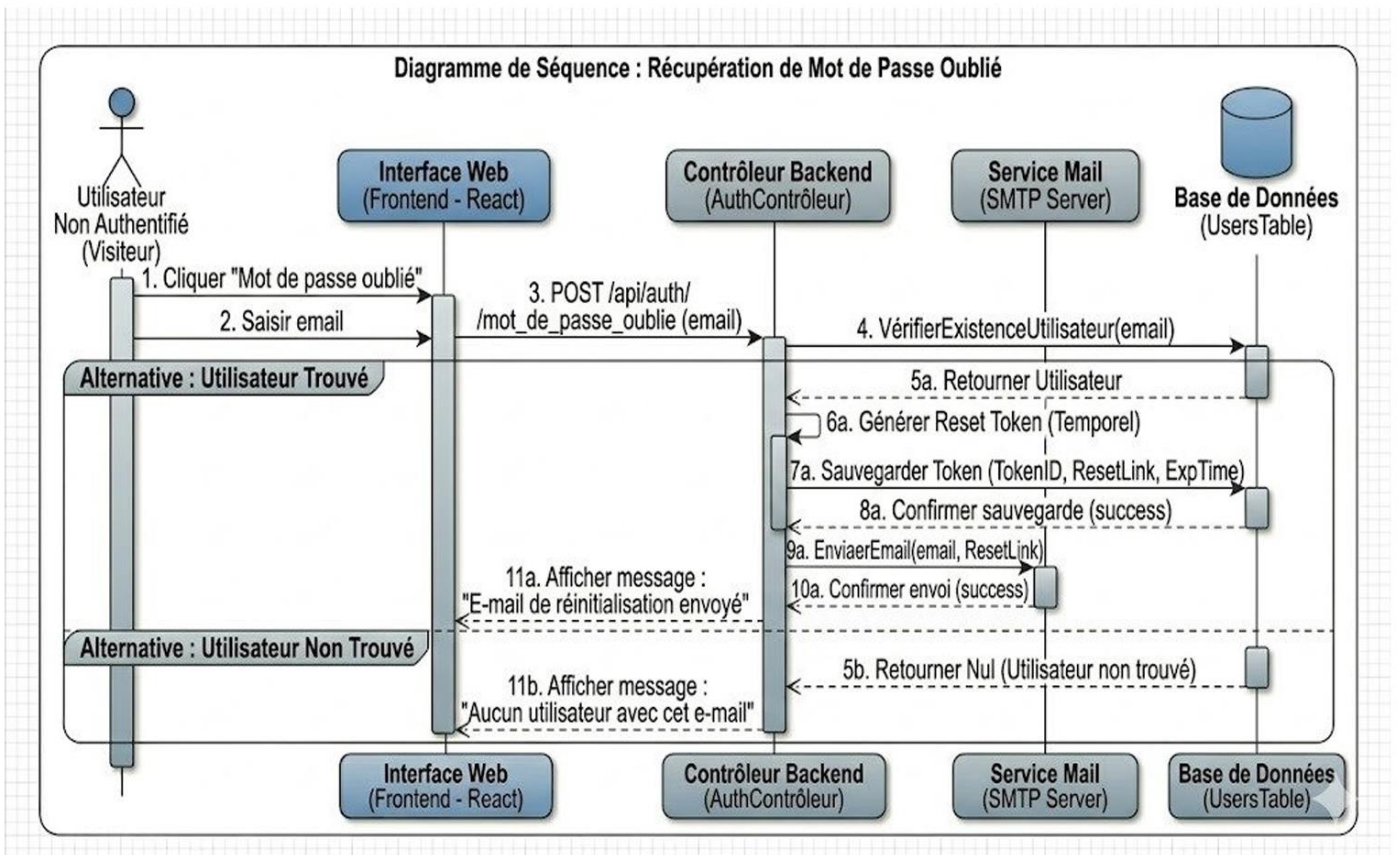


Figure 6 : Diagramme de Séquence - Récupération de Mot de Passe

4. Modélisation de la Base de Données

4.1. Modèle Conceptuel de Données (MCD)

Le modèle conceptuel de données (MCD) représente les principales entités du système E-Store ainsi que les relations qui existent entre elles. Cette modélisation permet d'avoir une vision claire de l'organisation des données et des interactions entre les différents composants du système.

Relations principales :

- **User** $\longleftrightarrow$ **Address** : relation *One-to-Many*
- Un utilisateur peut posséder plusieurs adresses de livraison.
- **User** $\longleftrightarrow$ **Cart** : relation *One-to-One*

Chaque utilisateur possède un seul panier.

- **Cart** $\longleftrightarrow$ **CartItem** : relation *One-to-Many*

Un panier peut contenir plusieurs articles.

- **CartItem** $\longleftrightarrow$ **Product** : relation *Many-to-One*

Plusieurs articles de panier peuvent référencer le même produit.

- **Category** $\longleftrightarrow$ **Product** : relation *One-to-Many*

Une catégorie peut contenir plusieurs produits.

- **Product** $\longleftrightarrow$ **Inventory** : relation *One-to-One*

Chaque produit possède un stock associé.

- **User** $\longleftrightarrow$ **Order** : relation *One-to-Many*

Un utilisateur peut effectuer plusieurs commandes.

- **Order** $\longleftrightarrow$ **OrderItem** : relation *One-to-Many*

Une commande contient plusieurs lignes de commande.

- **Product** $\longleftrightarrow$ **OrderItem** : relation *One-to-Many*

Un produit peut apparaître dans plusieurs commandes.

4.2. Schéma Logique MySQL

Le schéma logique MySQL regroupe les tables utilisées pour gérer les données transactionnelles du système. Les tables sont normalisées afin de réduire les redondances, éviter les anomalies de mise à jour et garantir l'intégrité référentielle.

Tables principales :

- **users** : stocke les informations des utilisateurs

(email, mot de passe, nom, etc.)

- **addresses** : contient les adresses de livraison associées aux utilisateurs.
- **categories** : regroupe les catégories de produits.
- **products** : représente le catalogue des produits avec leurs informations

(prix, description, images, etc.)

- **inventories** : gère le stock de chaque produit.
- **carts** : représente le panier associé à chaque utilisateur.
- **cart_items** : contient les articles présents dans le panier

(produit et quantité)

- **orders** : stocke les commandes avec le montant total, le statut et l'adresse de livraison.
- **order_items** : contient les lignes de commande

(produit, quantité et prix)

- **user_roles** : gère les rôles des utilisateurs

(USER, ADMIN)

4.3. Collections MongoDB

La base MongoDB est utilisée pour stocker les avis des utilisateurs sur les produits grâce à une structure flexible adaptée aux données semi-structurées.

Collection : reviews

Cette collection contient les informations suivantes :

- **_id** : identifiant unique MongoDB (*ObjectId*)
- **productId** : référence au produit dans MySQL
- **userId** : référence à l'utilisateur dans MySQL
- **userName** : nom de l'utilisateur (*dénormalisation pour optimisation*)
- **rating** : note attribuée au produit (*de 1 à 5 étoiles*)
- **title** : titre de l'avis
- **comment** : commentaire textuel (*jusqu'à 1000 caractères*)
- **images** : tableau contenant les images associées à l'avis
- **helpful** : nombre d'utilisateurs ayant trouvé l'avis utile
- **verified** : indique si l'achat est vérifié
- **tags** : liste de tags permettant de catégoriser l'avis
- **createdAt** : date de création
- **updatedAt** : date de dernière modification

Index MongoDB :

Afin d'optimiser les performances des requêtes, plusieurs index sont mis en place :

- Index sur **productId** pour les recherches par produit
- Index sur **userId** pour les recherches par utilisateur
- Index descendant sur **createdAt** pour afficher les avis récents
- Index sur **rating** pour le filtrage par note

4.4. Conclusion du Chapitre

Ce chapitre a présenté la conception architecturale complète du système **E-Store**. L'architecture trois-tiers découplée, combinée à une organisation en couches du backend ainsi qu'à une persistance hybride utilisant MySQL et MongoDB, offre une base solide pour le développement d'une application maintenable, évolutive et performante.

Les différents diagrammes UML ainsi que le schéma détaillé de la base de données constituent un véritable blueprint technique pour la phase d'implémentation. Cette conception claire permet aux développeurs de partager une vision commune du système et de travailler de manière cohérente et coordonnée.

Les prochains chapitres présenteront l'implémentation technique du projet en utilisant les technologies sélectionnées, notamment **Spring Boot**, **React.js**, **MySQL** et **MongoDB**, ainsi que la manière dont cette architecture est concrètement appliquée dans le code source.

CHAPITRE 4 : ENVIRONNEMENT TECHNIQUE ET DÉVELOPPEMENT

1. Architecture Technique Générale

Le projet **E-Store** a été développé selon une architecture moderne trois-tiers (Three-Tier Architecture), permettant une séparation claire entre :

- La couche présentation (**Frontend React.js**)
- La couche métier (**Backend Spring Boot**)
- La couche persistance (**MySQL & MongoDB**)

Cette architecture découplée facilite :

- la maintenance du code,
- l'évolutivité de l'application,
- les tests indépendants,
- et le développement collaboratif.

L'ensemble des échanges entre le Frontend et le Backend s'effectue via des APIs REST sécurisées utilisant le format JSON.

2. Stack Backend : Spring Boot & Java

2.1. Présentation de Spring Boot

Spring Boot est un framework Java permettant de développer rapidement des applications web robustes et prêtes pour la production.

Il simplifie considérablement :

- la configuration,
- la gestion des dépendances,
- la création d'APIs REST,
- et l'intégration avec les bases de données.

Dans le projet E-Store, Spring Boot a été utilisé pour :

- gérer les produits,
- les utilisateurs,
- les commandes,
- le panier,
- le stock,

- et les opérations d'administration.

2.2. Avantages de Spring Boot

Spring Boot offre plusieurs avantages majeurs :

- Configuration automatique des composants
- Serveur Tomcat intégré
- Gestion simplifiée des dépendances avec Maven
- Création rapide d'APIs RESTful
- Intégration native avec Hibernate et Spring Data JPA
- Support des profils de configuration
- Architecture modulaire et maintenable

Grâce à ces fonctionnalités, le développement du backend a été fortement accéléré tout en conservant une architecture professionnelle.

2.3. Java 17

Le projet a été développé avec Java, version LTS (Long Term Support) stable et optimisée.

Java 17 apporte :

- de meilleures performances,
- une sécurité renforcée,
- une meilleure gestion mémoire,
- ainsi qu'une syntaxe modernisée.

Cette version est largement utilisée dans les applications professionnelles modernes.

2.4. Architecture Backend

Le backend suit une architecture en couches basée sur le pattern MVC :

Couche	Rôle
Controller	Gestion des requêtes HTTP
Service	Logique métier
Repository	Accès aux données
Entity	Modélisation des tables

DTO Transfert des
données

Cette séparation améliore :

- la lisibilité du code,
- la maintenabilité,
- les tests,
- et la modularité.

2.5. Spring Data JPA

Spring Data JPA a été utilisé pour simplifier l'accès aux données relationnelles.

Exemple :

```
public interface ProductRepository extends JpaRepository<Product,  
Long> {  
  
    List<Product> findByNameContaining(String name);  
  
}
```

Avantages :

- génération automatique des opérations CRUD,
- requêtes dérivées des noms de méthodes,
- pagination intégrée,
- gestion simplifiée des relations entre entités.

2.6. Hibernate ORM

Hibernate est utilisé comme ORM (Object Relational Mapping).

Il permet :

- le mappage automatique entre objets Java et tables SQL,
- le Lazy Loading,
- la gestion automatique des transactions,
- la synchronisation des entités avec la base de données.

2.7. Configuration du Backend

Exemple de configuration :

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/estore_db
    username: root
    password: root

  jpa:
    hibernate:
      ddl-auto: update

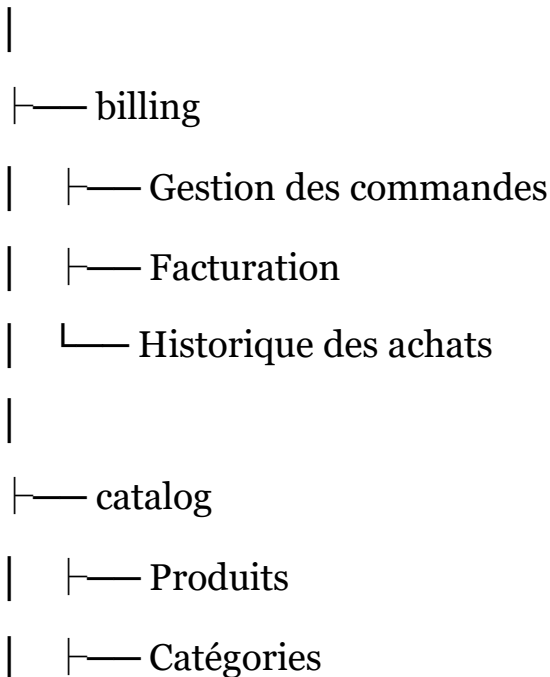
  data:
    mongodb:
      uri: mongodb://localhost:27017/estore_reviews
```

Cette configuration permet :

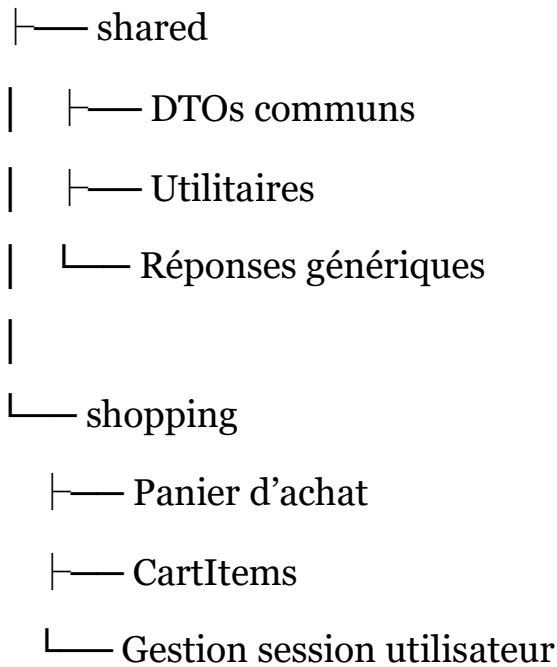
- la connexion à MySQL,
- la gestion JPA,
- ainsi que l'intégration de MongoDB.

2.8. Structure du Backend

com.estore



- | └─ Catalogue e-commerce
- |
- | └─ config
- | └─ Configuration Spring Boot
- | └─ CORS
- | └─ Beans globaux
- |
- | └─ customer
- | └─ Utilisateurs
- | └─ Authentification
- | └─ Profils clients
- |
- | └─ exception
- | └─ Gestion centralisée des erreurs
- | └─ Exceptions métier
- | └─ Validation API
- |
- | └─ inventory
- | └─ Gestion des stocks
- | └─ Synchronisation des quantités
- |
- | └─ review
- | └─ Avis clients
- | └─ Notes produits
- | └─ MongoDB
- |



3. Stack Frontend : React.js

3.1. *Présentation de React*

React est une bibliothèque JavaScript utilisée pour construire des interfaces utilisateur dynamiques et réactives.

React repose sur :

- les composants réutilisables,
- le Virtual DOM,
- et une architecture déclarative.

Dans E-Store, React a été utilisé pour :

- le catalogue produits,
- le panier,
- l'authentification,
- les commandes,
- et le panneau d'administration.

3.2. *Concepts Utilisés*

Components

Chaque partie de l'interface est un composant indépendant :

- ProductCard
- CartPage
- Header
- AdminPage
- LoginPage

Cette modularité facilite :

- la réutilisation,
- la maintenance,
- et l'évolution du projet.

3.3. Hooks React

useState

Gestion des états dynamiques :

```
const [products, setProducts] = useState([]);
```

useEffect

Chargement automatique des données :

```
useEffect(() => {  
    fetchProducts();  
}, []);
```

3.4. React Router

React Router permet la navigation entre les pages sans rechargement complet.

Exemples :

- /products
- /cart
- /orders
- /admin

3.5. Axios

Axios a été utilisé pour communiquer avec les APIs Spring Boot.

Exemple :

```
const { data } = await api.get('/products');
```

Axios facilite :

- les requêtes HTTP,
- la gestion des tokens,
- et les appels asynchrones.

3.6. Responsive Design

Le responsive design a été implémenté avec CSS Grid et Flexbox.

Exemple :

```
.products-grid {  
  display: grid;  
  grid-template-columns: repeat(4, 1fr);  
  gap: 20px;  
}
```

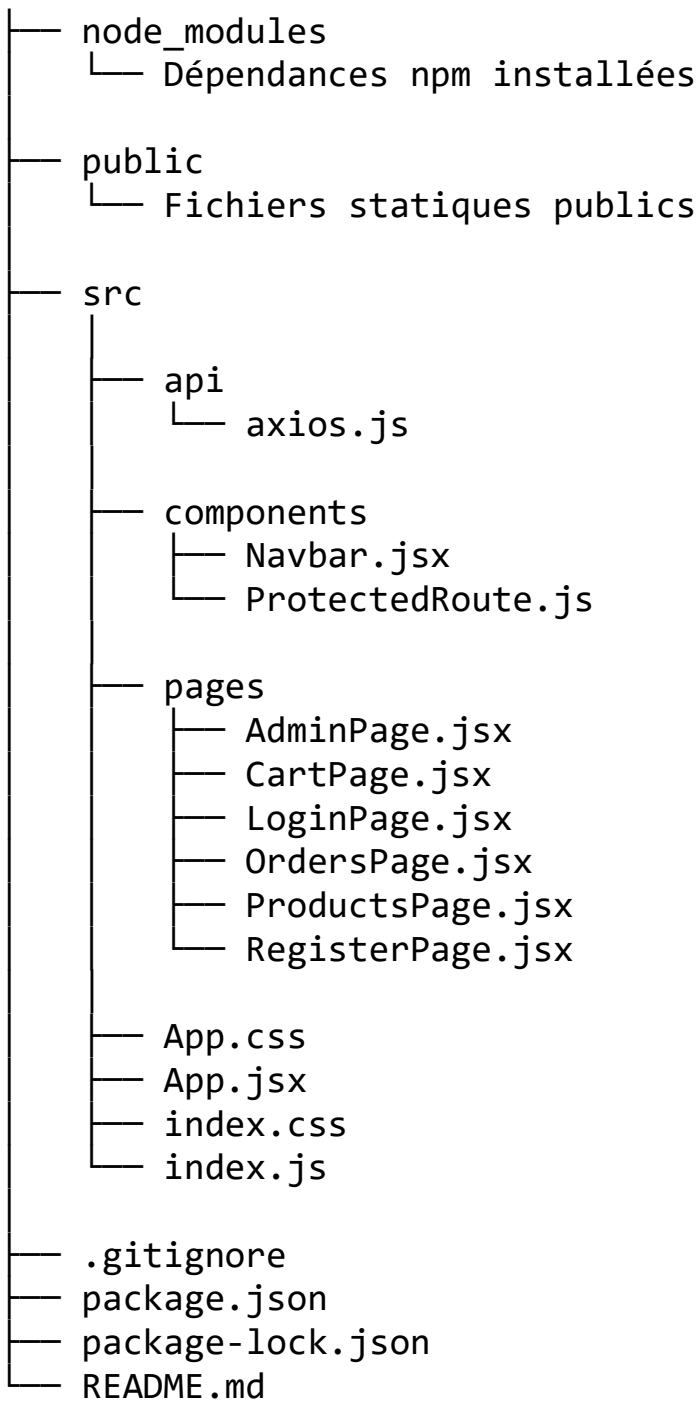
Responsive :

```
@media (max-width: 768px) {  
  grid-template-columns: repeat(2, 1fr);  
}
```

Cette approche garantit :

- une adaptation mobile,
- tablette,
- et desktop.

3.7. Structure Frontend



4. Outils de Développement

4.1. IntelliJ IDEA

IntelliJ IDEA a été utilisé pour le développement backend.

Fonctionnalités :

- autocomplétion,
- débogage,

- gestion Maven,
- inspections automatiques.

4.2. Visual Studio Code

Visual Studio Code a été utilisé pour React.js.

Avantages :

- extensions React,
- terminal intégré,
- Git intégré,
- rapidité d'exécution.

4.3. Postman

Postman a servi à tester toutes les APIs REST.

Utilisations :

- validation des endpoints,
- tests manuels,
- vérification des réponses JSON,
- détection des erreurs.

4.4. Git & GitHub

Git et [GitHub](#) ont permis :

- le versionnement,
- la sauvegarde du code,
- le suivi des modifications,
- et la collaboration.

CHAPITRE 5 : GESTION DES DONNÉES — MYSQL ET MONGODB

1. Approche Polyglot Persistence

Le projet E-Store utilise une stratégie appelée **Polyglot Persistence**, consistant à combiner plusieurs systèmes de bases de données selon leurs avantages respectifs.

Deux technologies ont été utilisées :

Technologie	Utilisation
MySQL	Données transactionnelles
MongoDB	Avis clients

Cette approche permet d'optimiser :

- les performances,
- la flexibilité,
- et la scalabilité.

2. MySQL : Gestion des Données Relationnelles

2.1. Pourquoi MySQL ?

MySQL a été choisi pour :

- sa fiabilité,
- son support ACID,
- ses relations fortes,
- et sa stabilité transactionnelle.

Il gère :

- utilisateurs,
- produits,
- commandes,
- panier,
- stock.

2.2. Tables Principales

Le schéma relationnel contient notamment :

- users
- products
- categories
- inventories
- carts
- cart_items
- orders
- order_items

2.3. Transactions

Les commandes utilisent les transactions Spring :

```
@Transactional  
public OrderResponse createOrder(CreateOrderRequest request)
```

Cela garantit :

- cohérence des données,
- rollback automatique en cas d'erreur,
- protection contre les incohérences.

2.4. Optimisation avec les Index

Des index ont été ajoutés afin d'améliorer les performances.

Exemple :

```
CREATE INDEX idx_products_category  
ON products(category_id);
```

Résultat :

- temps réduit,
- recherches accélérées,
- meilleures performances globales.

3. MongoDB : Gestion des Avis Clients

3.1. Pourquoi MongoDB ?

MongoDB a été utilisé pour les avis clients.

MongoDB est adapté aux données :

- flexibles,
- variables,
- évolutives.

3.2. Exemple de Document

```
{  
  "productId": 12,  
  "userName": "Jean",  
  "rating": 5,  
  "comment": "Excellent produit"  
}
```

3.3. Avantages

MongoDB permet :

- une structure flexible,
- des agrégations rapides,
- une grande scalabilité,
- et une meilleure gestion des données non relationnelles.

CHAPITRE 6 : TESTS, VALIDATION ET DÉBOGAGE

1. Tests des APIs avec Postman

Toutes les APIs REST ont été validées avec Postman.

Les tests ont permis de :

- vérifier les routes,
- tester les réponses JSON,
- détecter les erreurs,
- valider les opérations CRUD.

2. Problèmes Rencontrés

Plusieurs difficultés techniques ont été rencontrées durant le développement.

Erreur 422

Cause :

- validation DTO incorrecte.

Solution :

- correction des champs JSON,
- ajout des annotations nécessaires.

Images manquantes dans le panier

Cause :

- image non copiée dans le DTO.

Solution :

- ajout du champ imageUrl dans CartItemDTO.

Erreurs CORS

Cause :

- communication React ↔ Spring bloquée.

Solution :

```
@Configuration
public class CorsConfig {
}
```

Concurrence sur le stock

Deux commandes simultanées pouvaient modifier le stock incorrectement.

Solution :

- utilisation du Pessimistic Locking.

3. Tests d'Intégration

Des tests complets ont été réalisés :

- ajout panier,
- validation commande,
- décrémentation stock,
- historique commandes.

Ces tests garantissent la cohérence globale de l'application.

CHAPITRE 7 : SÉCURITÉ, PERFORMANCES ET PERSPECTIVES

1. Sécurité

1.1. JWT Authentication

JWT permet l'authentification sécurisée.

Fonctionnement :

1. login utilisateur,
2. génération token JWT,
3. stockage côté client,
4. vérification à chaque requête.

1.2. Hashage des Mots de Passe

Les mots de passe sont hashés avec BCrypt :

```
new BCryptPasswordEncoder().encode(password)
```

Cela empêche le stockage des mots de passe en clair.

1.3. Validation des Données

Des validations ont été ajoutées :

@NotBlank
@Email
@Size

Ces validations protègent :

- les données,
- la sécurité,
- et l'intégrité applicative.

2. Optimisations de Performance

Plusieurs optimisations ont été mises en place :

- Index SQL
- Lazy Loading
- Pagination
- Optimisation des requêtes
- Responsive Design
- Réduction des appels inutiles

3. Perspectives d'Évolution

Le projet peut évoluer avec :

- paiement Stripe,
- sécurité Spring Security + JWT complète,
- notifications email,
- dashboard analytique,
- système de recommandations,

- architecture microservices,
- déploiement Docker/Kubernetes,
- application mobile.

CONCLUSION GÉNÉRALE

Le projet **E-Store** représente une plateforme e-commerce Full Stack moderne développée selon les standards professionnels actuels.

Ce projet nous a permis d'appliquer concrètement :

- les architectures modernes,
- le développement frontend/backend,
- les APIs REST,
- la gestion des bases de données,
- la sécurité,
- et les bonnes pratiques du génie logiciel.

L'utilisation combinée de :

- Spring Boot,
- React.js,
- MySQL,
- MongoDB,
- et GitHub

a permis de construire une application :

- modulaire,
- scalable,
- maintenable,
- et évolutive.

Ce projet constitue une expérience complète couvrant l'ensemble du cycle de développement d'une application web moderne.

Merci pour votre attention et pour le temps
consacré à l'évaluation de ce travail.

Nous espérons que ce projet E-Store reflète les
compétences techniques et les connaissances
acquises tout au long de notre parcours
universitaire.